



Introduction to Node.js

single-threaded event loop model

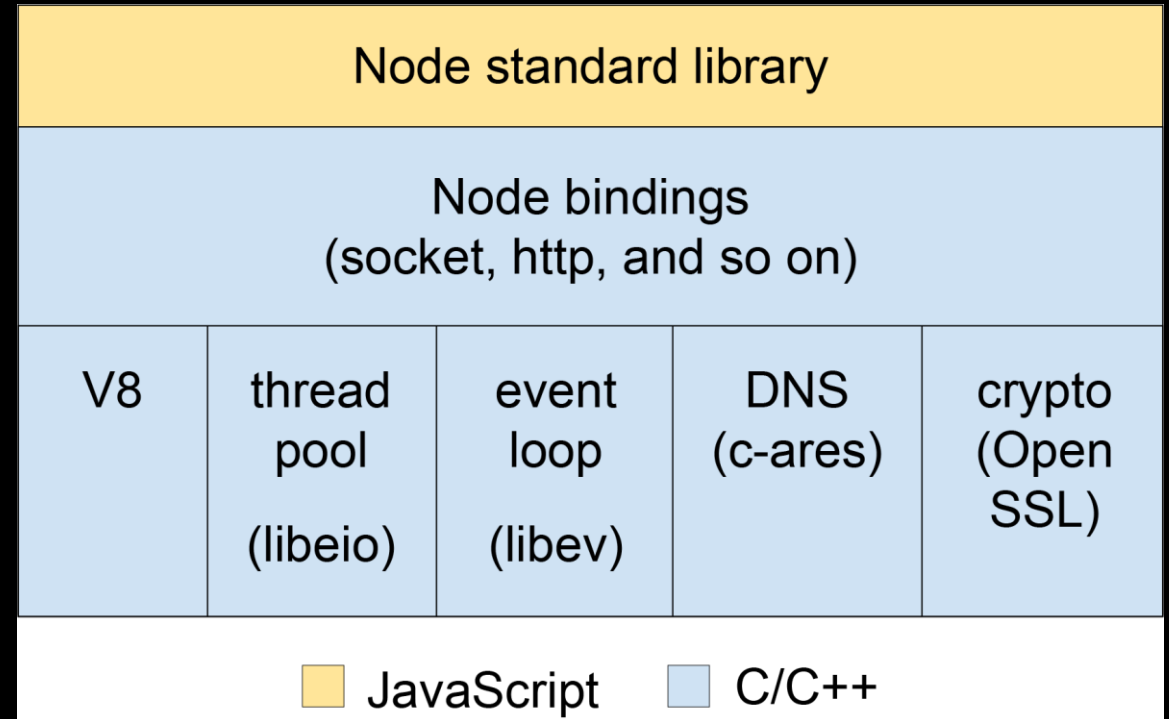


เป้าหมาย

- Describe the basics of how Node.js works
- List the applications of Node.js in modern software development
- Describe the module system used by Node.js
- Implement basic modules for an application
- Explain the asynchronous programming basics in Node.js
- Implement a basic application using async/await

Node.js is an event-driven, server-side JavaScript environment.

- Node.js runs as a single-threaded process that acts upon callbacks and never blocks on the main thread, making it high-performing for web applications.



Node.js ใช้ทำอะไร?



Creating REST APIs



Creating real-time services



Building microservices



Tooling

Module System

- organizing code, hide information, and only expose the public interface of a component using `module.exports`.

```
// math.js file
function add(a, b)
{
    return a + b;
}

module.exports = { add, mul, div,};
```

```
// index.js file
const math = require('./math');
console.log(math.add(30, 20)); // 50
```

โมดูล มี 3 ประเภท

Built-in (native)

modules: โมดูลติดตั้งมาพร้อมกับ Node.js ใช้ได้เลย

Third-party modules:

โมดูลที่ต้องติดตั้งเพิ่มโดยใช้ npm หรือเครื่องมือติดตั้งอื่น

Local modules: โมดูลสร้างไว้ใช้เองภายในองค์กร

Node Package Manager (npm) is the package manager for JavaScript and the world's largest software registry, enabling developers to discover packages of reusable code.

NPM

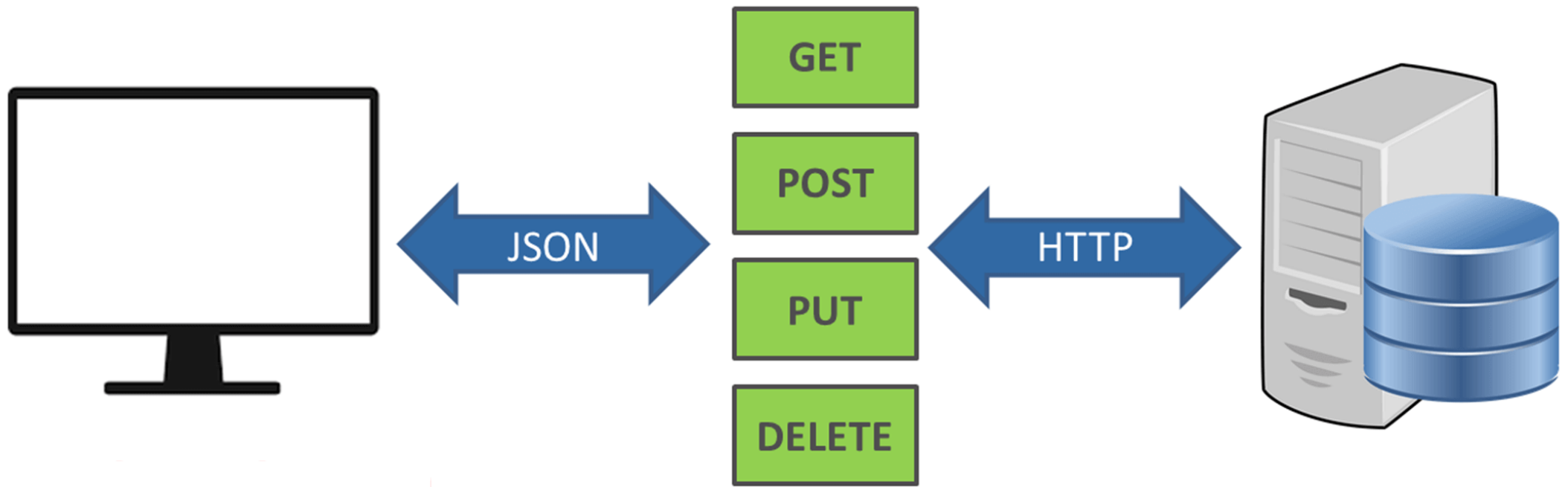
- Node Package Manager (npm) is the package manager for JavaScript and the world's largest software registry, enabling developers to discover packages of reusable code.

เซิร์ฟเวอร์ ในวิชานี้ คือ ซอฟต์แวร์ที่คอยให้บริการ

API Paradigms (กรอบการทำงาน)

REST, RPC, GraphQL, WebHooks, WebSocket, OpenAPI

Request-response APIs vs. Event-driven APIs



This Photo by Unknown Author is licensed under CC BY-SA-NC

Request-Response APIs (สื่อสารโดย ร้องขอ-ตอบสนอง)

- เปิดให้บริการ โดยโปรแกรมอื่นติดต่อผ่านทาง HTTP-based web server
 - API กำหนดจุดปลายทางสำหรับเข้าถึงข้อมูล (endpoints) ไว้รอให้บริการ
 - โปรแกรมไคลเอนต์ ส่ง http request ไปยัง endpoints นั้น
 - เซิร์ฟเวอร์จะส่งข้อมูล หรือบริการกลับมาในรูปแบบของ JSON หรือ XML
- Paradigms เช่น REST, RPC, OpenAPI และ GraphQL เป็นแบบนี้

REST: Representational State Transfer

ได้รับความนิยมมาก มีผู้บริการเอพีไอรูปแบบนี้หลายเจ้า เช่น Google, Stripe, Twitter และ GitHub



C - Create
R - Read
U - Update
D - Delete

เอ็นทีทีอะไรก็ตามที่สามารถ
ระบุตัว, ตั้งชื่อ, กำหนดที่อยู่ และถูกจัดการผ่านเว็บได้
เช่น ลูกค้า, บัญชีเงินฝาก, รหัสไปรษณีย์, สินค้า

- Resources เป็นส่วนหนึ่งของ URL เช่น /users
- ใช้คำนาม ระบุ resources
- แต่ละ resource มักจะสร้างสองชุด เป็นชื่อ collection กับระบุค่าใดค่าหนึ่ง เช่น /std กับ /std/6509610012
- ใน Request ใช้ HTTP Methods เช่น GET, POST, UPDATE, DELETE ในการสั่งให้เซิร์ฟเวอร์จัดการข้อมูล

Building RESTful APIs with Node.js and Express

Babel : เครื่องมือสำหรับคอมไพล์ โค้ดที่ใช้ไวยากรณ์ใหม่ เป็น JavaScript เวอร์ชันที่เบราว์เซอร์เข้าใจ

Express: JS library

MongoDB: ระบบจัดการฐานข้อมูลเพื่อใช้เก็บรักษาข้อมูลให้คงไว้ (Persist) ได้

เตรียมสร้างเอพีไอ

- สร้างโฟลเดอร์ crm เข้าไปในโฟลเดอร์นั้น แล้วสั่ง npm init เพื่อสร้าง package.json
- Install express ด้วยคำสั่ง npm i express
- Install MongoDB Community (<https://www.mongodb.com/try/download/community>)
- ติดตั้ง Mongoose เป็นไลบรารีช่วยสร้างโมเดลของฐานข้อมูล, มีเครื่องมือช่วยเช็ค validate ข้อมูล

```
npm i --save-dev @babel/core @babel/cli @babel/node @babel/preset-env
```

```
npm i nodemon body-parser
```

- MacOS ควรติดตั้ง Homebrew ไว้สำหรับ install program

เริ่มต้นรันเซิร์ฟเวอร์

- ไฟล์ package.json กำหนดให้ script ให้เริ่มต้น start server program

“start”: “nodemon ./index.js --exec babel-node”

และระบุ package type เป็น โมดูล

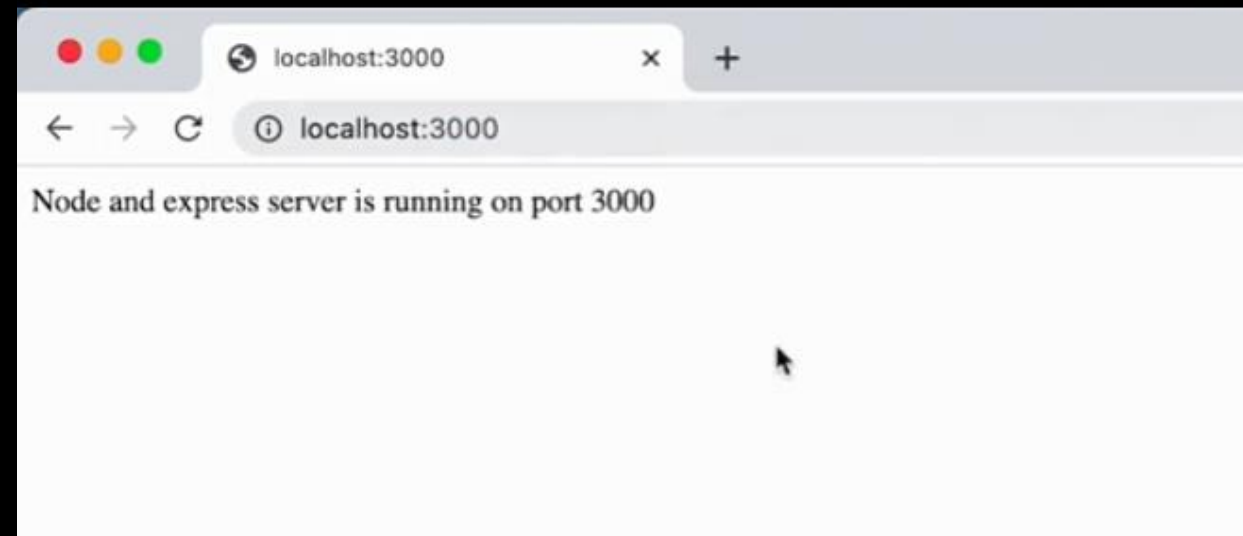
“type”: “module”

- เขียนโค้ดใน index.js โดยใช้ express ให้รองรับ request จาก client
- สตาร์ทเซิร์ฟเวอร์ ด้วย คำสั่ง

npm start

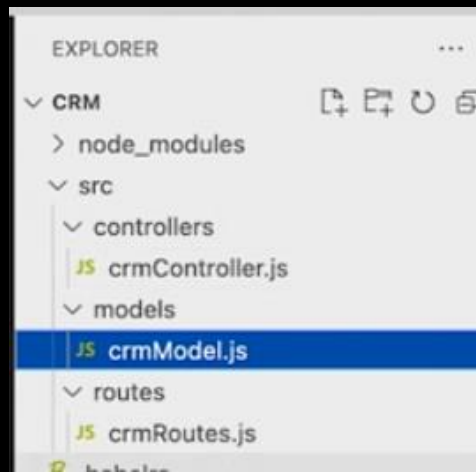
- ทดลองว่าเซิร์ฟเวอร์ทำงานแล้ว จากเว็บเบราว์เซอร์ โดยส่ง request ไปที่ localhost:3000

```
JS index.js > ...
1  import express from 'express';
2
3  const app = express();
4  const PORT = 3000;
5
6  app.get('/', (req, res) =>
7    |   res.send(`Node and express server is running on port ${PORT}`)
8  );
9
10 app.listen(PORT, () =>
11   |   console.log(`Your server is running on port ${PORT}`)
12 );
13
```

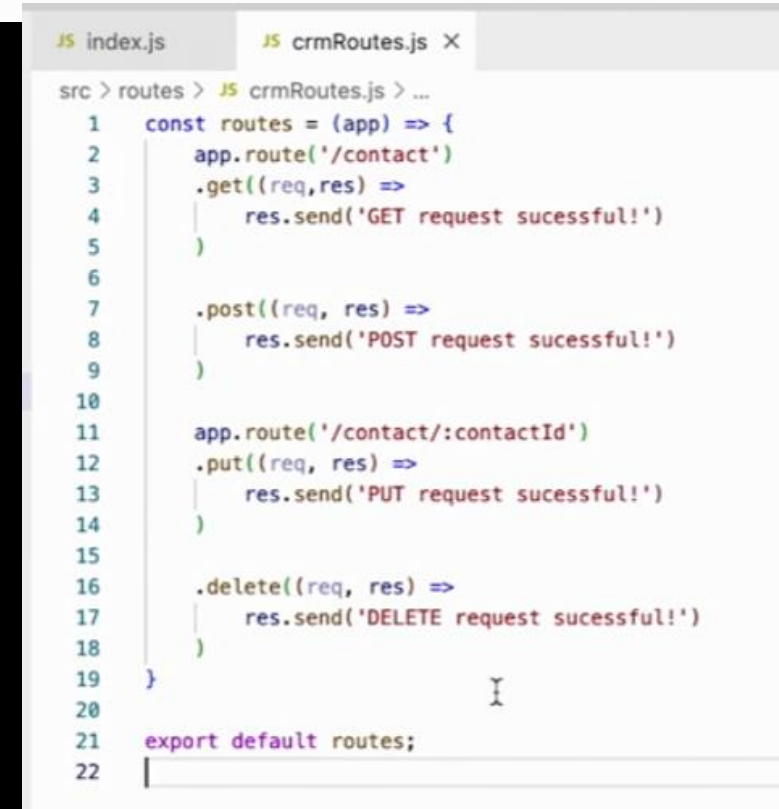


การตั้งเส้นทางไปยังเอ็นพอยต์ (จุดปลายทางบริการ)

CH2>02_03



- controllers (ฟังก์ชันที่เราจะนำข้อมูลไปใส่เอ็นพอยต์และส่งต่อไปให้ใครที่ต้องการ)
- models (โมเดลการออกแบบฐานข้อมูล schema models)
- routes (คือ endpoint)
- ที่ package root folder สร้างไฟล์เดอร์ย่อย ดังรูปขวามือ
 - กำหนดให้ index.js ส่ง app (express object) ไปให้ route.js ควบคุม
 - สร้าง endpoints ต่าง ๆ ใน route.js
 - ทดลองส่ง GET request ไปยัง localhost:3000/contact



Middleware

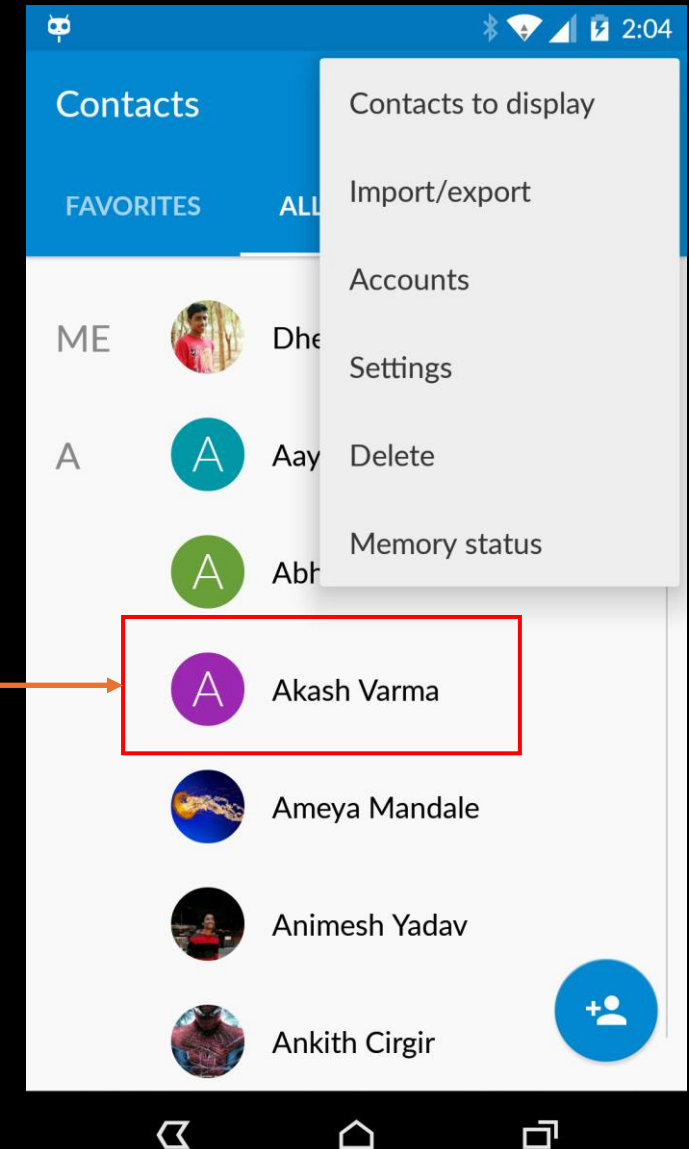
- ฟังก์ชันที่อยู่ระหว่าง request กับ response
- middleware ใช้จัดการสิ่งต่างๆ ก่อนที่จะส่ง next เพื่อส่งต่อไป response
-

```
JS index.js  JS crmRoutes.js ●
src > routes > JS crmRoutes.js > [e] routes
1  const routes = (app) => {
2    app.route('/contact')
3    .get((req, res, next) => {
4      // middleware
5      console.log(`Request from: ${req.originalUrl}`)
6      console.log(`Request type: ${req.method}`)
7      next();
8    }, (req, res, next) => {
9      res.send('GET request sucessful!')
10   })
11
12   .post((req, res) =>
13     res.send('POST request sucessful!')
14   )
15
16   app.route('/contact/:contactId')
17   .put((req, res) =>
18     res.send('PUT request sucessful!')
19   )
20
21   .delete((req, res) =>
22     res.send('DELETE request sucessful!')
23   )
24 }
25
```

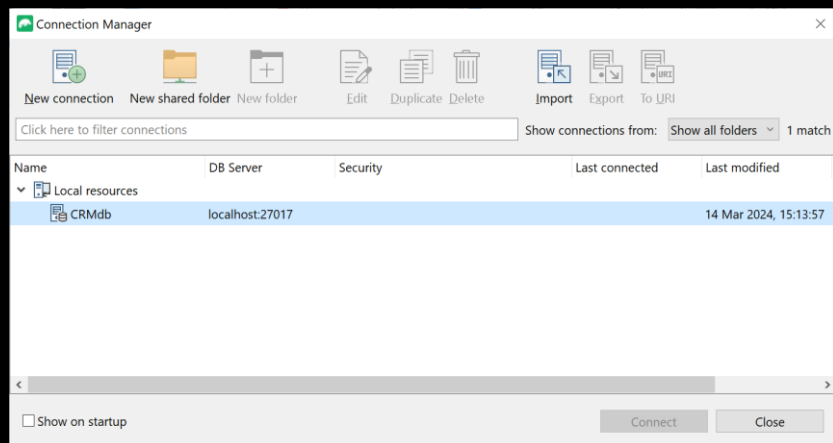
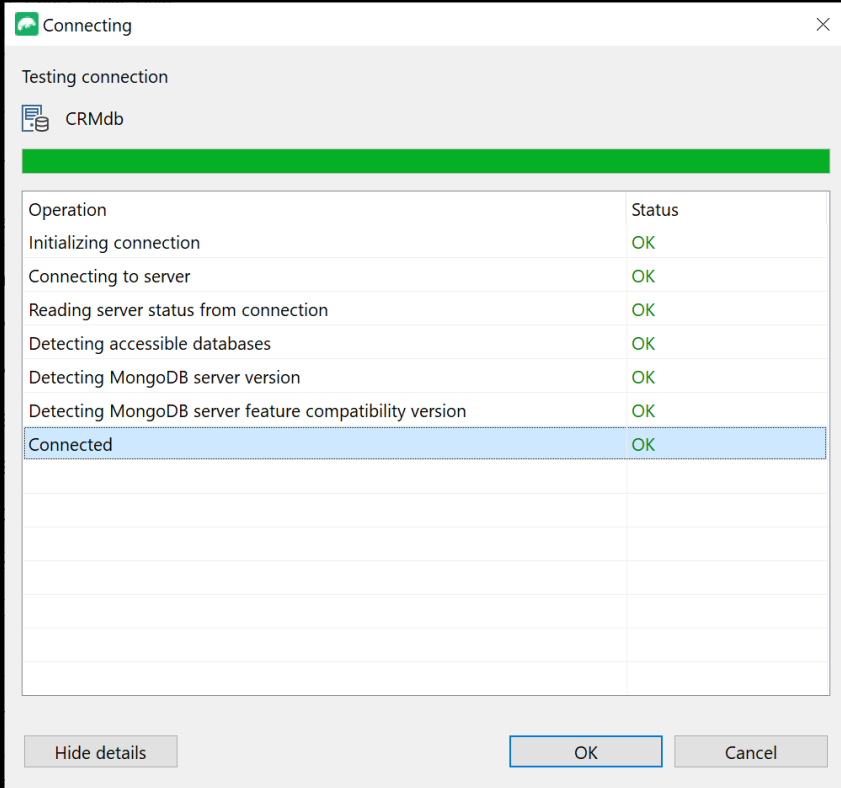
MongoDB

- ซอฟต์แวร์จัดการฐานข้อมูล
- ฐานข้อมูล (Database) ประกอบด้วย collections
- แต่ละ Collection ประกอบด้วย เอกสาร (Documents) หรือ Objects
- แต่ละ Document จะเก็บ ข้อมูล ในรูปแบบ คู่ key-value, หรือ arrays ของ ไอ์

แต่ละ Contact เป็น
a Document



Contact Database เป็น a Collection



จัดเตรียมฐานข้อมูล

- ติดตั้ง Studio 3T
 - เลือก Connect สร้าง New Connection
 - ใช้ default password encryption
 - ใช้ localhost
 - พอร์ตตั้งต้นของ MongoDB
 - ตั้งชื่อ ฐานข้อมูลว่า CRMdb
 - ทดลองเชื่อมต่อ

เชื่อมต่อกับฐานข้อมูล

- ที่ไฟล์ index.js

```
JS index.js  X
JS index.js > ...
1  import express from 'express';
2  import mongoose from 'mongoose';
3  import bodyParser from 'body-parser';
4  import routes from './src/routes/crmRoutes';
5
6  const app = express();
7  const PORT = 3000;
8
9  // mongoose connection
10 mongoose.Promise = global.Promise;
11 mongoose.connect('mongodb://localhost/CRMdb',{
12   |   useNewUrlParser: true
13   | });
14
15 //bodyparser setup
16 app.use(bodyParser.urlencoded({ extended: true }));
17 app.use(bodyParser.json());
18
19
20 routes(app);
21
22 app.get('/', (req, res) =>
23   |   res.send(`Node and express server is running on port ${PORT}`)
24   | );
25
```

สร้าง Schema

- สคีมา เป็นโครงที่บอกว่าแต่ละ document หรือแต่ละ Object จะมีรายละเอียดอะไรบ้าง
- ตัวอย่างเช่น Contact แต่ละคนจะเก็บ ชื่อ, สกุล, email, ฯลฯ เป็นต้น
- ที่โมเดล `crmModel.js` สร้าง schema ของฐานข้อมูลดังรูป

JS crmModel.js ●

src > models > JS crmModel.js > [🔗] ContactSchema > 🔑 created_date

```
1  import mongoose from 'mongoose';
2
3  const Schema = mongoose.Schema;
4
5  export const ContactSchema = new Schema({
6    firstName: {
7      type: String,
8      required: 'Enter a first name'
9    },
10   lastName: {
11     type: String,
12     required: 'Enter a last name'
13   },
14   email: {
15     type: String
16   },
17   company: {
18     type: String
19   },
20   phone: {
21     type: Number
22   },
23   created_date: {
24     type: Date,
25     default: Date.now
26   }
27 })
```

CRUD

JS crmController.js

src > controllers > JS crmController.js > addNewContact

```
1 import mongoose from 'mongoose';
2 import { ContactSchema } from '../models/crmModel';
3
4 const Contact = mongoose.model('Contact', ContactSchema);
5
6 export const addNewContact = (req, res) => {
7   let newContact = new Contact(req.body);
8
9   newContact.save((err, contact) => {
10     if (err) {
11       res.send(err);
12     }
13     res.json(contact);
14   });
15 }
```

JS crmController.js

JS crmRoutes.js X

src > routes > JS crmRoutes.js > routes

```
1 import { addNewContact } from '../controllers/crmController';
2
3 const routes = (app) => {
4   app.route('/contact')
5     .get((req, res, next) => {
6       // middleware
7       console.log(`Request from: ${req.originalUrl}`)
8       console.log(`Request type: ${req.method}`)
9       next();
10     }, (req, res, next) => {
11       res.send('GET request sucessful!')
12     })
13
14     .post(addNewContact);
15
16   app.route('/contact/:contactId')
17     .put((req, res) => {
18       res.send('PUT request sucessful!')
19     })
20
21     .delete((req, res) => {
22       res.send('DELETE request sucessful!')
23     })
24
25
26   export default routes;
27 }
```

localhost:3000/contact

POST localhost:3000/contact

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL

KEY	VALUE	DESCRIPTION	***	Bulk Edit
☒ firstName	Manny			
☒ lastName	Henri			
☒ company	Linkedin			
☒ phone	5555551234			
Key	Value	Description		

Studio 3T for MongoDB - Full product trial

Search Open Connections (XF) ...

localhost:27017 [direct]

CRMDB

Collections (1)

contacts

GridFS Buckets (0)

System (0)

Views (0)

admin

config

local

Collections (1)

startup_log

GridFS Buckets (0)

System (0)

Views (0)

contacts

Query

Projection

Skip

Sort

Limit

Result

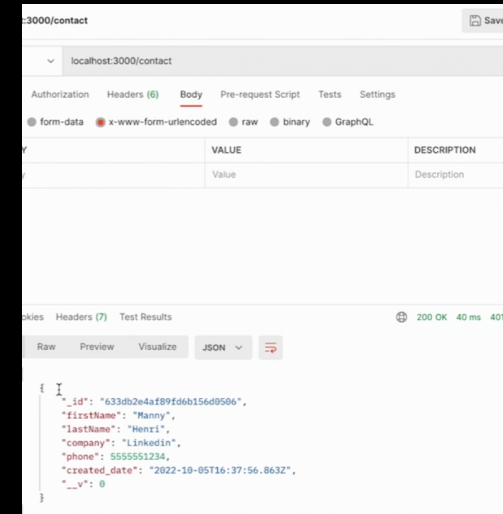
Query Code

Explain

Documents 1 to 1

_id	firstName	lastName	company	phone	created_date	...
633db2e4af99	Manny	Henri	Linkedin	5555551234.0	2022-10-05T16:11:00	

สร้าง GET Endpoint เพื่อ ดึงข้อมูลทุก Contact



localhost:3000/contact

Authorization Headers (6) Body Pre-request Script Tests Settings

form-data x-www-form-urlencoded raw binary GraphQL

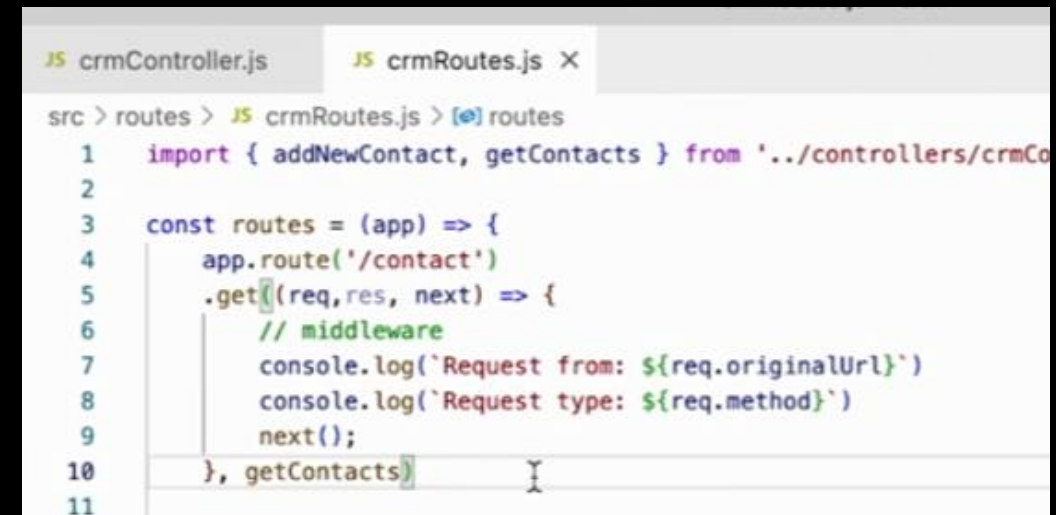
NAME	VALUE	DESCRIPTION
	Value	Description

200 OK 40 ms 401 B

```
{
  "_id": "633db2e4af89fd6b156d9566",
  "firstName": "Manny",
  "lastName": "Henri",
  "company": "LinkedIn",
  "phone": 5555551234,
  "created_date": "2022-10-05T16:37:56.863Z",
  "___v": 0
}
```



```
JS crmController.js JS crmRoutes.js
src > controllers > JS crmController.js > ...
1 import mongoose from 'mongoose';
2 import { ContactSchema } from '../models/crmModel';
3
4 const Contact = mongoose.model('Contact', ContactSchema);
5
6 export const addNewContact = (req, res) => {
7   let newContact = new Contact(req.body);
8
9   newContact.save((err, contact) => {
10     if (err) {
11       res.send(err);
12     }
13     res.json(contact);
14   });
15 };
16
17 export const getContacts = (req, res) => {
18   Contact.find({}, (err, contact) => {
19     if (err) {
20       res.send(err);
21     }
22     res.json(contact);
23   });
24 }
25
```



```
JS crmController.js JS crmRoutes.js X
src > routes > JS crmRoutes.js > routes
1 import { addNewContact, getContacts } from '../controllers/crmCo
2
3 const routes = (app) => {
4   app.route('/contact')
5     .get((req, res, next) => {
6       // middleware
7       console.log(`Request from: ${req.originalUrl}`)
8       console.log(`Request type: ${req.method}`)
9       next();
10     }, getContacts);
11
```

สร้าง GET Endpoint แบบค้นหาจาก ID

```
JS crmController.js X JS crmRoutes.js
src > controllers > JS crmController.js > [e] getContactWithID
12     }
13     res.json(contact);
14   });
15 };
16
17 export const getContacts = (req, res) => {
18   Contact.find({}, (err, contact) => {
19     if (err) {
20       res.send(err);
21     }
22     res.json(contact);
23   });
24 };
25
26 export const getContactWithID = (req, res) => {
27   Contact.findById(req.params.contactId, (err, contact) => {
28     if (err) {
29       res.send(err);
30     }
31     res.json(contact);
32   });
33 };
34
```

```
JS crmController.js JS crmRoutes.js X
src > routes > JS crmRoutes.js > [e] routes
1  import { addNewContact, getContacts, getContactWithID } from '../controllers/crmController';
2
3  const routes = (app) => {
4    app.route('/contact')
5      // get all contacts
6      .get((req, res, next) => {
7        // middleware
8        console.log(`Request from: ${req.originalUrl}`)
9        console.log(`Request type: ${req.method}`)
10       next();
11     }, getContacts)
12
13     // post a new contact
14     .post(addNewContact);
15
16     app.route('/contact/:contactId')
17       // get specific contact
18       .get(getContactWithID)
19
20       .put((req, res) => {
21         res.send('PUT request sucessful!')
22       })
23
24       .delete((req, res) => {
25         res.send('DELETE request sucessful!')
26       })
27   }
28
29   export default routes;
30
```

สร้าง PUT Endpoint

```
JS crmController.js • JS crmRoutes.js
src > controllers > JS crmController.js > [0] updateContact > Contact.findOneAndUpdate() callback
12 |     }
13 |     res.json(contact);
14 |   });
15 | };
16 |
17 | export const getContacts = (req, res) => {
18 |   Contact.find({}, (err, contact) => {
19 |     if (err) {
20 |       res.send(err);
21 |     }
22 |     res.json(contact);
23 |   });
24 | };
25 |
26 | export const getContactWithID = (req, res) => {
27 |   Contact.findById(req.params.contactId, (err, contact) => {
28 |     if (err) {
29 |       res.send(err);
30 |     }
31 |     res.json(contact);
32 |   });
33 | };
34 |
35 | export const updateContact = (req, res) => {
36 |   Contact.findOneAndUpdate({ _id: req.params.contactId }, req.body, { new: true }, (err, contact) => {
37 |     if (err) {
38 |       res.send(err);
39 |     }
40 |     res.json(contact);
41 |   });
42 | };
43 |
```

```
JS crmController.js JS crmRoutes.js X
src > routes > JS crmRoutes.js > [0] routes
1 | import { addNewContact, getContacts, getContactWithID, updateContact } from '../controllers/crmController';
2 |
3 | const routes = (app) => {
4 |   app.route('/contact')
5 |     // get all contacts
6 |     .get((req, res, next) => {
7 |       // middleware
8 |       console.log('Request from: ${req.originalUrl}')
9 |       console.log('Request type: ${req.method}')
10 |       next();
11 |     }, getContacts)
12 |
13 |     // post a new contact
14 |     .post(addNewContact);
15 |
16 |   app.route('/contact/:contactId')
17 |     // get specific contact
18 |     .get(getContactWithID)
19 |
20 |     // update a contact
21 |     .put(updateContact);
22 |
23 |     .delete((req, res) => {
24 |       res.send('DELETE request successful!');
25 |     })
26 |   }
27 |
28 |   export default routes;
29 |
```

สร้าง DELETE Endpoint

```
JS crmController.js X JS crmRoutes.js ●
src > controllers > JS crmController.js > ...
34
35 export const updateContact = (req, res) => {
36   Contact.findOneAndUpdate({ _id: req.params.contactId }, req.body, { new: true }, (err, contact) => {
37     if (err) {
38       res.send(err);
39     }
40     res.json(contact);
41   });
42 };
43
44 export const deleteContact = (req, res) => {
45   Contact.remove({ _id: req.params.contactId }, (err) => {
46     if (err) {
47       res.send(err);
48     }
49     res.json({ message: 'Successfully deleted contact' });
50   });
51 };
52
53
```

```
JS crmController.js JS crmRoutes.js ●
src > routes > JS crmRoutes.js > [0] routes
1 import { addNewContact,
2   getContacts,
3   getContactWithID,
4   updateContact,
5   deleteContact } from '../controllers/crmController';
6
7 const routes = (app) => {
8   app.route('/contact')
9     // get all contacts
10    .get((req, res, next) => {
11      // middleware
12      console.log('Request from: ${req.originalUrl}')
13      console.log('Request type: ${req.method}')
14      next();
15    }, getContacts)
16
17    // post a new contact
18    .post(addNewContact);
19
20    app.route('/contact/:contactId')
21    // get specific contact
22    .get(getContactWithID)
23
24    // update a contact
25    .put(updateContact)
26
27    // to delete a contact
28    .delete(deleteContact)
29
30
31 export default routes;
32
```